

Exhaustive Analysis of Firewall Architectures: Packet Identification, Stateful Mechanics, and Hybrid Control Systems

Introduction to Modern Perimeter Defense and Security Orchestration

The conceptual architecture of network firewalls has evolved from rudimentary, static packet-filtering routers into highly sophisticated, context-aware security enforcement platforms. At its core, a firewall serves as a cryptographic and logical boundary, isolating an organization's internal infrastructure from external threat vectors and untrusted networks. However, the proliferation of cloud-native computing architectures, heavily encrypted tunneling protocols, and highly distributed enterprise perimeters has fundamentally altered the structural and computational requirements of these security appliances. A modern firewall is no longer merely a static gateway that blindly permits or denies traffic based on human-configured lists; rather, it is an intelligent, automated orchestration engine capable of real-time protocol analysis, dynamic rule generation, and seamless integration with broader global threat intelligence networks.

To satisfy the requirements of a standard, enterprise-grade firewall today, an organization must deploy a system that balances deterministic manual control with highly responsive automated mitigation. Network administrators require granular manual control to architect the physical and logical topology of the network, establish baseline security policies, configure complex Network Address Translation (NAT) rules, and manage the specific routing behavior of the appliance. Simultaneously, the velocity and sophistication of modern cyber threats—such as AI-orchestrated exploit toolkits and massive distributed denial-of-service (DDoS) campaigns—necessitate automated control mechanisms. These automated systems must be capable of ingesting external threat feeds, dynamically modifying access control lists, and deploying immediate countermeasures without requiring human intervention.

This comprehensive report elucidates the exhaustive technical principles underpinning firewall operation. It examines the computational mechanics of packet identification across the Open Systems Interconnection (OSI) model, the architectural topologies of deployment, rule evaluation algorithms, and the high-performance packet processing pipelines that enable line-rate throughput in modern data centers. Furthermore, it explores the critical synergy between manual administrative control and fully automated, API-driven threat response mechanisms—a hybrid operational paradigm unequivocally required to secure contemporary network ecosystems.

Core Principles of Packet Identification and Inspection

The fundamental operation of any firewall relies entirely on its ability to identify, parse, and evaluate network traffic against a defined security policy. This classification occurs across multiple layers of the OSI model, varying significantly in computational depth, memory utilization, and architectural complexity. The depth to which a firewall interrogates a packet determines its classification as a stateless filter, a stateful engine, or a next-generation deep packet inspection platform.

Stateless Inspection Mechanics and Header Parsing

Stateless firewalls operate as the most foundational tier of network defense, functioning as deterministic packet filters that evaluate each network frame in strict isolation.¹ Operating primarily at OSI Layers 2 through 4 (the Data Link, Network, and Transport layers), the stateless engine parses static protocol headers to make its routing or blocking decisions.² When a packet arrives at the network interface, the stateless engine examines attributes such as the source and destination IP addresses, the specific TCP or UDP port numbers, and specific control flags embedded within the protocol header.²

Because a stateless firewall fundamentally lacks an internal memory mechanism to track the broader context of a communication session, it relies entirely on predefined Access Control Lists (ACLs) to dictate action.³ When a packet enters the processing pipeline, the engine performs a direct linear or hashed lookup against this static policy table.³ Based on the programmed parameters, it makes a rigid, instantaneous decision to ALLOW, DENY, or RESET the connection.³ If the parsed packet parameters fall outside the exact boundaries of what is considered acceptable by the administrator or the manufacturer, the stateless protocol identifies the packet as a violation and restricts its transit.²

The primary architectural advantage of stateless inspection is its negligible computational overhead. Because the evaluation relies purely on static header data and does not require the operating system to allocate memory for massive state tables or track session histories, the stateless engine achieves line-rate processing with sub-millisecond latency.³ However, this architectural simplicity introduces severe operational limitations in complex environments. Stateless firewalls are inherently unidirectional in their logic.³ For bidirectional communication to function—such as an internal client making an outbound HTTP request and receiving the corresponding inbound reply from an external server—the network administrator must manually write two separate, static ACL rules.³ The administrator must whitelist both the outbound request path and the inbound return path.

This bidirectional requirement makes stateless architectures highly error-prone at scale, as human administrators must manage thousands of discrete rules. More critically, stateless firewalls are entirely incapable of securely handling complex protocols that dynamically negotiate ephemeral ports during runtime.³ For example, the File Transfer Protocol (FTP) establishes a static control connection on TCP Port 21, but it dynamically negotiates a random, high-numbered ephemeral port for the actual data transfer.³ Because a stateless firewall cannot dynamically inspect control payloads or adapt its ruleset on the fly, administrators are forced to leave massive ranges of high-numbered ports permanently open to accommodate

FTP data transfers, creating unacceptable security vulnerabilities.³

Stateful Inspection and Connection Tracking

To directly address the operational limitations and security vulnerabilities of stateless filtering, stateful inspection was developed and rapidly adopted as the foundational standard for enterprise firewalls.¹ A stateful firewall differs from a stateless one by maintaining a dynamic internal state table—often referred to as a flow table or a connection tracking engine—to monitor the entire lifecycle of a network session.¹ Rather than evaluating packets as isolated, independent entities, stateful engines inspect packets in the contextual reality of their broader traffic flow.¹

When an initial packet arrives at a stateful firewall, the engine performs a comprehensive 5-tuple lookup, comprising the source IP address, destination IP address, source port, destination port, and the protocol identifier.³ If this 5-tuple does not match any existing entry in the flow table, the packet is treated as a new connection and is routed to the "slow path" or control plane.³ In the slow path, the firewall performs a rigorous policy lookup against its security ruleset.³ If the overarching policy permits the connection, the firewall takes action, allowing the packet through and writing a new dynamic entry into the state flow table.³ Crucially, once this state is established, all subsequent packets that match this exact 5-tuple flow are aggressively routed through the firewall's "fast path" or data plane.³ Fast-path processing allows these subsequent packets to entirely bypass the computationally expensive slow-path policy lookup, relying instead on the established trust of the state table.³ This fast-path processing involves rapid rate checks, Layer 3 and Layer 4 sanitation checks to prevent spoofing or fragmentation attacks, and then immediate forwarding.³ This stateful awareness introduces a profound operational advantage: it enables the firewall to dynamically and automatically permit inbound return traffic associated with a legitimate outbound request.³ Administrators only need to write a single rule permitting the outbound traffic; the state engine automatically tracks the session and creates a temporary, dynamic allowance for the returning data, drastically reducing the complexity of manual rule configuration and tightening the perimeter security posture.³

Feature	Stateless Inspection Architecture	Stateful Inspection Architecture
OSI Layer Focus	Layers 2 - 4 (Packet Headers Only)	Layers 3 - 4 (Headers, Context, and Flow)
Memory Utilization	Negligible (No session tables required)	High (Requires RAM for massive flow tables)
Processing Velocity	Extremely high (Consistent)	Variable (Slower on initiation, faster on

	line-rate)	established flows)
Bidirectional Rule Management	Requires manual inbound and outbound rules	Automatic return traffic allowance via state tracking
Complex Protocol Support	None (Fails on dynamic/ephemeral ports)	High (Supports ALGs for dynamic port negotiation)

Deep Packet Inspection (DPI) and Next-Generation Capabilities

While stateful inspection solved the problem of dynamic port negotiation and bidirectional rule bloat, it eventually became insufficient against modern evasion techniques. As network architecture evolved, threat actors adapted by utilizing standard, universally permitted ports—such as TCP Port 80 for HTTP or TCP Port 443 for HTTPS—to tunnel malicious payloads into corporate networks. A traditional stateful firewall, seeing traffic moving over Port 443, would simply verify the TCP handshake and permit the flow, blind to the actual contents of the communication.⁷

Next-Generation Firewalls (NGFWs) address this critical vulnerability by extending their operational scope up to OSI Layer 7 (the Application Layer) through the implementation of Deep Packet Inspection (DPI).⁸ DPI transcends the rudimentary evaluation of network headers by explicitly interrogating the actual data payload carried within the packet.⁸ This capability grants the firewall "application awareness," allowing it to understand specific applications, services, and the intent of their traffic.⁸

With application awareness, an NGFW can differentiate between vastly different types of traffic that happen to traverse the same port.⁷ For example, a traditional firewall sees all traffic on Port 443 identically. An NGFW, utilizing DPI, can distinguish between a user browsing a standard corporate website, a user attempting to upload a proprietary document to a cloud storage application, or a concealed command-and-control (C2) beacon communicating with a threat actor's infrastructure—all operating over the exact same port.⁸ Consequently, administrators can implement granular, manual controls based on user identity, application type, and geographic location rather than simple IP addresses.⁸

Furthermore, modern NGFWs must integrate robust TLS/SSL decryption modules.⁷ Because contemporary malware and C2 communications are overwhelmingly encrypted, a firewall cannot perform DPI on a ciphertext payload. To inspect the data, the NGFW must function as a transparent intermediate certificate authority. It terminates the encrypted connection from the client, decrypts the traffic, inspects the plaintext payload via DPI for advanced threats, and then re-encrypts the data before forwarding it to the final destination.⁷ This decryption cycle is computationally immense. With the widespread adoption of modern encryption standards like TLS 1.3, the decryption overhead alone is enough to reduce a firewall's maximum throughput by a staggering 50% to 80% if the appliance lacks dedicated hardware cryptographic

accelerators.¹⁰

Advanced Protocol and State Tracking Mechanics

The internal mathematical and logical mechanics of stateful firewalls rely heavily on sophisticated connection tracking engines. The conntrack system, natively integrated into the Linux kernel's Netfilter framework, represents the definitive standard for understanding how firewalls manage session states across varying protocols.⁶

TCP State Machine Synchronization

For connection-oriented protocols such as the Transmission Control Protocol (TCP), the firewall's connection tracking engine precisely synchronizes its internal logic with the standard TCP three-way handshake.³

1. **Initiation (NEW):** When an internal client initiates a connection, it sends a packet with the SYN control bit set. The firewall receives this, performs a policy lookup, and logs the session in the flow table with an internal state of NEW.³
2. **Acknowledgment (ESTABLISHED):** When the external server responds with a packet containing the SYN + ACK flags, the firewall recognizes the reciprocal communication. It promotes the connection's internal state within the flow table to ESTABLISHED.³ This state indicates that the packet belongs to a connection that has witnessed valid bidirectional traffic.⁶
3. **Termination:** When the communication concludes, one of the hosts will send a FIN + ACK packet to gracefully close the connection, or a RST packet to forcefully terminate it. Upon detecting these specific control bits, the firewall marks the corresponding state table entry for deletion.³ Any future packets arriving that attempt to leverage this defunct flow are immediately dropped, preventing attackers from hijacking stale sessions.³

Pseudo-State Generation for Connectionless Protocols

While TCP provides explicit control flags for state tracking, connectionless protocols like the User Datagram Protocol (UDP) and the Internet Control Message Protocol (ICMP) present a distinct challenge. Because these protocols lack inherent sequencing, handshakes, or teardown mechanisms, the firewall must mathematically generate a "pseudo-state" to track them.³

For UDP traffic (commonly used by DNS, DHCP, SNMP, and TFTP), the firewall analyzes the outbound packet and extracts the source IP, destination IP, source port, and destination port.³ It records this data in the connection table and starts a dynamically expiring timeout timer.³ The firewall holds this pseudo-state open only for the duration of the timeout window. If an inbound UDP packet arrives that perfectly matches the reverse 5-tuple before the timer expires, the firewall permits the packet as a valid reply. If the timer expires, the state is purged, and subsequent inbound packets are silently dropped.³

ICMP tracking operates on a similar pseudo-state principle but requires the firewall to parse the internal ICMP header types and codes.¹⁴ Network diagnostic tools like ping and traceroute

rely heavily on ICMP messages.¹⁴ When an internal device executes a ping, it sends an ICMP Type 8 (Echo Request) message.¹⁴ The firewall tracks the unique ICMP identifier and sequence number embedded in the payload.¹⁴ The returning ICMP Type 0 (Echo Reply) is only permitted through the firewall if it precisely matches the expected sequence and identifier of the generated pseudo-state.¹⁴

Furthermore, network infrastructure heavily relies on ICMP error messages. For instance, if a router cannot deliver a packet, it will drop the packet and send back an ICMP Type 3 (Destination Unreachable) or a Type 11 (Time Exceeded) message.¹⁴ Advanced connection tracking engines examine the payload of these ICMP error messages, extract the original IP header of the failed packet, and map it back to the original UDP or TCP flow that triggered the error.¹²

The Application Layer Gateway and the RELATED State

The mapping of associated traffic introduces another critical state within the Linux conntrack model: the RELATED state.⁶ While the NEW state identifies fresh connections and ESTABLISHED identifies ongoing bidirectional flows, the RELATED state is utilized when a packet initiates a new connection that is inextricably linked to an existing, established flow.⁶

This mechanism is vital for securely managing complex protocols like FTP or SIP. As previously noted, FTP utilizes a control channel on Port 21 but negotiates data transfers over dynamic ports.³ A stateful firewall employs an Application Layer Gateway (ALG) to deeply inspect the FTP control payload in real-time.³ When the ALG observes the client and server negotiating a new, random port for a file transfer, the firewall dynamically injects a temporary rule into the state table, flagging the anticipated data channel as RELATED.³ When the data transfer initiates on that random port, the firewall recognizes it as RELATED to the original Port 21 session and permits it, safely bypassing the overarching default-deny policy without requiring administrators to leave wide port ranges open permanently.³

Conversely, any packet that cannot be associated with a known connection and does not conform to expected protocol behavior—such as an unsolicited ACK packet arriving without a preceding SYN—is categorized as INVALID.⁶ Standard firewall best practices dictate that manual rules must explicitly drop all INVALID packets to thwart network reconnaissance mapping and TCP state-exhaustion attacks.⁶

Structural Architectures and Network Topologies

The physical placement and logical configuration of a firewall fundamentally dictate its efficacy. Network architects must evaluate the organization's risk tolerance, throughput requirements, and existing infrastructure to determine the optimal deployment topology.

Segmented Deployment Topologies

1. **Bastion Host Topology:** The bastion host is a highly secured, heavily hardened system deliberately exposed to an untrusted network, typically the public internet.¹⁹ This architecture minimizes its attack surface by disabling all non-essential services, operating

system daemons, and unnecessary user accounts.¹⁹ Placed directly between the internet and the internal network segment, the bastion host serves as the singular gateway for external access.²⁰ Because it is a rich, prominent target for external attacks, it must be meticulously maintained and monitored.²⁰

2. **Dual-Homed Host Architecture:** Building upon the bastion host concept, a dual-homed firewall incorporates at least two distinct Network Interface Cards (NICs) within the chassis.¹⁹ One NIC connects exclusively to the external untrusted network, while the other connects to the internal secure network.²⁰ This topology enforces strict logical and physical separation. By disabling IP forwarding at the operating system level, traffic cannot natively route between the two NICs; it is forced to traverse the firewall's inspection engine.²⁰ This architecture frequently utilizes Network Address Translation (NAT) to map external IP addresses to internal ones, completely obscuring the internal network topology from external observers.²⁰
3. **Screened Subnet (DMZ) Architecture:** Considered the most resilient of traditional network architectures, the screened subnet introduces a Demilitarized Zone (DMZ) flanked by two distinct firewalls.¹⁹ The external firewall filters public traffic attempting to access the DMZ, which houses publicly accessible systems such as web servers, email gateways, and external DNS servers.¹⁹ The internal firewall strictly isolates the DMZ from the corporate backend intranet.¹⁹ This design severely limits lateral movement; even if a sophisticated threat actor successfully compromises a web server within the DMZ, they remain trapped within the screened subnet and must defeat a second, distinct internal firewall to reach sensitive corporate databases.¹⁹

Transparent vs. Routed Operational Modes

Beyond physical topology, firewalls can be deployed in radically different logical operating modes regarding how they handle IP routing. The choice between routed and transparent modes determines how the firewall integrates into the existing network fabric.

Routed Mode (Layer 3 Operation): In routed firewall mode, the security appliance acts as an active, visible participant in the network's Layer 3 infrastructure.²² Each interface on the firewall is assigned a unique IP address on a distinct subnet, and the firewall explicitly routes traffic between these subnets just as a standard network router would.²³ This mode supports complex dynamic routing protocols—such as OSPF, RIP, and BGP—allowing the firewall to learn and advertise routes across the enterprise.²³ Furthermore, a routed firewall can act as the default gateway for internal devices, centralizing both routing and security in a single appliance.²³ The primary drawback of routed mode is deployment complexity; introducing a routed firewall into an existing network inevitably requires extensive IP re-addressing, subnet restructuring, and routing table modifications.²²

Transparent Mode (Layer 2 Bridging): Conversely, a transparent firewall (often referred to as a bridging firewall) operates silently at Layer 2 of the OSI model, effectively acting as an invisible "bump in the wire".²² In this configuration, the firewall's inside and outside interfaces are grouped together into a Bridge Group, and the firewall utilizes standard bridging

techniques to pass traffic between the interfaces without decrementing the IP Time-to-Live (TTL) counter or altering MAC addresses.²³ The bridge group is managed via a Bridge Virtual Interface (BVI), which is assigned an IP address solely for management purposes, not for active routing.²³

Because the transparent firewall lacks a routed IP presence on the specific user subnets it protects, it is virtually invisible to attackers performing network reconnaissance or running traceroute commands.²² Transparent deployments are heavily favored when an organization must rapidly introduce deep security inspection into an established, complex network without undergoing the highly disruptive and lengthy process of IP re-addressing.²² Furthermore, transparent firewalls possess the unique capability to permit or filter non-IP traffic protocols—such as AppleTalk, IPX, and Bridge Protocol Data Units (BPDUs)—which are natively dropped by Layer 3 routed appliances.²²

Deployment Mode	OSI Operation Level	Network Visibility	Deployment Disruption	Routing Capabilities
Routed Mode	Layer 3 (Network)	Highly visible as a network hop	High (Requires significant IP re-addressing)	Supports static routing, OSPF, BGP
Transparent Mode	Layer 2 (Data Link)	Invisible "bump in the wire"	Low (Rapid drop-in deployment)	Relies on BVI bridging, no dynamic routing

Circuit-Level Gateways and Proxy Architectures

In addition to standard stateful packet filtering, organizations often deploy specialized gateway architectures to further obfuscate internal topology and mandate strict protocol compliance.

Circuit-Level Gateways (Layer 5 Operation): Operating primarily at the session layer of the OSI model, a circuit-level gateway firewall monitors and validates the establishment of TCP and UDP connections before any data is allowed to be forwarded.²⁷ Rather than deeply inspecting the data payload like an NGFW, it intercepts the client's connection request, rigorously validates the TCP three-way handshake, and then establishes a secondary, entirely separate connection on behalf of the client to the external server.²⁷

This splicing mechanism creates a virtual circuit.²⁹ By acting as a relay, the gateway ensures that the external server only ever interacts with the gateway's IP address, completely preserving the anonymity of the internal hosts.²⁸ The SOCKS (Socket Secure) protocol is the de facto industry standard for implementing circuit-level proxies.³⁰ As defined in RFC 1928, SOCKS Version 5 facilitates the seamless and secure traversal of firewalls for both TCP and UDP traffic, providing strong authentication mechanisms while forwarding traffic without deep payload

inspection.³⁰

Application-Level Gateways / Proxy Firewalls (Layer 7 Operation): While circuit-level gateways splice the connection at the session layer, application proxy firewalls provide the most stringent form of traditional firewalling by fully terminating the connection at the application layer.³² An application proxy prevents internal networks from ever directly contacting external systems.³² When a client makes a request, the proxy terminates the connection, deeply inspects the payload, validates it against specific application RFC standards, and then recreates the request to the destination server.³² By acting as this absolute intermediary, it guarantees that traffic strictly adheres to expected protocol behavior—for instance, ensuring HTTP traffic does not contain malformed headers aiming to trigger buffer overflows.³² However, the requirement to completely buffer, decrypt, inspect, and reconstruct every single session introduces significant latency, severely impacting network speed compared to circuit-level or stateful architectures.³²

Rule Evaluation Engines and Policy Processing Paradigms

The deterministic reliability of a firewall relies completely on its internal rule evaluation engine. The specific sequence in which manual access control rules are parsed and evaluated dictates both the security posture and the computational efficiency of the appliance. A poorly structured ruleset can inadvertently bypass intended security controls, lock administrators out of the system, or severely degrade throughput.³⁴ Firewall operating systems are broadly divided into two foundational parsing philosophies.

The "First Match Wins" Paradigm

In a "first match wins" architecture—most prominently utilized by the Linux iptables framework, Cisco Access Control Lists (ACLs), and many commercial enterprise firewalls—the engine evaluates rules sequentially from the top of the list to the bottom.³⁵ The instant a packet's attributes match the criteria defined in a rule, the corresponding action (e.g., ACCEPT or DROP) is immediately executed, and rule processing halts entirely for that packet.³⁵

This approach necessitates a strict, logical, and hierarchical configuration strategy.³⁵ Administrators must prioritize exclusivity. Highly specific exception rules (e.g., "Allow Host 10.0.0.5 to access Port 80") must be placed at the top of the ruleset, followed by increasingly general rules, culminating in a broad, "catch-all" default-deny rule at the absolute bottom.³⁴ If an administrator mistakenly places the default deny rule first, it will match all incoming traffic, the evaluation will halt, and the subsequent specific exception rules will never be evaluated, resulting in a systemic access failure across the network.³⁴

The "Last Match Wins" Paradigm

Conversely, the OpenBSD pf (Packet Filter) engine—the underlying technology powering renowned open-source firewall distributions like pfSense and OPNsense—defaults to a radically different "last match wins" paradigm.³⁶ When a packet enters a pf firewall, the engine

parses the entire ruleset from top to bottom, continually evaluating every rule.³⁶ If a rule matches, the engine temporarily notes the action, but it does not stop. It continues down the list, constantly overwriting its internal decision as subsequent matching rules are encountered. The packet's ultimate fate is decided by the action of the *final* matching rule in the entire list.³⁶ This architectural choice allows administrators to construct highly readable, declarative policies.³⁶ Broad, sweeping default policies (e.g., block drop all) are established at the very top of the configuration file. Specific, granular exceptions are then layered progressively beneath them.³⁶ Because the engine evaluates everything, administrators do not have to worry about a broad rule at the top prematurely halting the evaluation of an exception placed below it.³⁶ However, evaluating every rule for every packet is computationally wasteful. To optimize performance and circumvent this behavior when necessary, pf introduces the quick keyword.³⁶ Appending quick to a rule instantly converts that specific rule to a "first match" evaluation; if the packet matches a quick rule, processing halts immediately, and the action is taken.³⁶ This keyword is critically deployed for early-exit drop rules—such as blocking traffic from known malicious botnet IPs at the absolute top of the ruleset. Using quick prevents these malicious packets from wasting CPU cycles that would otherwise be spent evaluating them against hundreds of subsequent rules.³⁶

Action-Order and Strict Priority Evaluation

Advanced packet evaluation engines, such as Suricata (frequently integrated into AWS Network Firewall and complex open-source deployments), implement rule evaluation based on action-group priority rather than strict top-down ordering.⁴⁰ By default, Suricata evaluates all pass rules before it assesses any drop, reject, or alert rules.⁴⁰ This internal priority mechanism ensures that explicitly whitelisted traffic is processed efficiently and is never inadvertently subjected to complex, CPU-intensive payload signature checks that belong to the drop or alert groups.⁴⁰

For complex enterprise deployments requiring absolute manual control, administrators can override this default behavior and enforce "strict ordering".⁴⁰ Under strict ordering, rule groups are processed exclusively by manually assigned priority integers (ranging from 1 to 65535), regardless of whether the action is a pass or a drop, ensuring absolute deterministic control over the firewall's decision matrix.⁴⁰

High-Performance Packet Processing Pipelines: Bypassing the Kernel

As enterprise network throughput scales beyond 10 gigabits per second (Gbps) and into the hundreds of Gbps, the traditional software architecture of operating systems becomes the primary performance bottleneck for firewalls. Specifically, the architectural divide between user space and kernel space creates unacceptable latency.⁴¹

The Context Switching and Memory Bottleneck

Historically, when network packets arrive at a physical Network Interface Card (NIC), the

hardware generates an interrupt.⁴² This interrupt forces the system CPU to pause its current tasks, switch context from user space to kernel space, and manage the reception of the packet.¹⁸ The Linux kernel must then dynamically allocate memory structures—specifically the `sk_buff` (socket buffer) data structure—to hold the packet data.¹⁸

The kernel parses the Ethernet, IP, and TCP headers, evaluates the packet against the iptables or nftables framework within the kernel's network stack, and finally copies the data to user space for application-level processing.¹⁸ The continuous, high-frequency context switching between kernel space and user space, combined with the heavy memory allocation overhead of generating an `sk_buff` for every single packet, creates severe latency. In environments subject to volumetric DDoS attacks or serving high-frequency trading platforms, this traditional interrupt-driven architecture will cause the CPU to max out, resulting in massive packet drops and system unavailability.¹⁸

DPDK and eBPF/XDP Architectures

To address this existential bottleneck, modern high-performance firewall pipelines leverage hardware-accelerated kernel bypass techniques and highly programmable data paths.

eBPF and Native XDP (eXpress Data Path): The extended Berkeley Packet Filter (eBPF) revolutionized Linux networking by allowing secure, sandboxed C-code to be compiled and executed directly within the Linux kernel without requiring changes to the kernel source code.¹⁸ When combined with XDP, this filtering logic is pushed down to the absolute lowest layer of the software network stack: the NIC device driver itself.¹⁸

In a firewall architecture utilizing Native XDP, an arriving packet is intercepted and evaluated by an eBPF program *before* it reaches the main TCP/IP networking stack, and crucially, *before* the kernel allocates any `sk_buff` memory.¹⁸ The eBPF program manually parses the raw packet headers.¹⁸ If the firewall logic determines the packet is part of a volumetric DDoS attack, a SYN flood, or originates from a blacklisted IP, the program issues an `XDP_DROP` action.¹⁸ The packet is immediately discarded at the driver level, resulting in virtually zero CPU overhead and completely preventing system memory exhaustion.¹⁸ If the packet is legitimate, the program issues an `XDP_PASS` command, allowing the packet to proceed up the standard kernel stack for deeper stateful analysis.¹⁸ By eliminating memory allocation for dropped packets, XDP allows software firewalls to mitigate massive flood attacks at hardware-level speeds.¹⁸

DPDK (Data Plane Development Kit): Alternatively, DPDK takes a different approach by bypassing the Linux kernel entirely.⁴³ Instead of relying on hardware interrupts, DPDK utilizes Poll Mode Drivers (PMDs) that run in user space.⁴⁷ These drivers constantly poll the NIC's hardware ring buffers, pulling packets directly into user-space memory.⁴³ By eliminating interrupts and context switching entirely, DPDK achieves theoretical maximum throughput for packet processing.⁴² However, bypassing the kernel means the firewall application must manually reimplement the entire TCP/IP stack (routing tables, ARP, TCP reassembly) in user space, vastly increasing software complexity.⁴⁷ Consequently, modern high-performance software firewalls increasingly favor eBPF/XDP for rapid ingress dropping and routing, while utilizing user-space programs solely for the "control plane"—managing dynamic threat maps,

compiling rules, and gathering telemetry data.¹⁸

Processing Architecture	Execution Location	Memory Overhead	Primary Use Case / Benefit
Traditional (iptables / nftables)	Kernel Space	High (Requires sk_buff allocation for every packet)	Standard stateful filtering and complex NAT operations ⁴⁴
DPDK	User Space	Low (Direct memory access via polling)	High-throughput specialized virtual network functions (VNFs) ⁴⁷
eBPF / XDP	NIC Driver Layer	Zero (for dropped packets)	Unmatched DDoS mitigation, pre-stack packet dropping at line rate ¹⁸

Manual Control versus Automated Orchestration Systems

A hallmark of a comprehensive, standard firewall deployment is the symbiotic integration between manual administrative control and automated, programmatic response. The complexities of the modern threat landscape—characterized by polymorphic malware, automated exploit toolkits that adapt in minutes, and rapidly shifting adversary infrastructure—render purely manual management entirely obsolete.¹¹ However, automated systems cannot architect network topologies; they must work in concert with deliberate human engineering.

Deterministic Manual Control

Manual control remains the bedrock of structural network security. Administrators must retain manual authority to configure the foundational policies that govern the network's existence. This includes:

- Establishing baseline default-deny postures and defining specific, business-justified ingress and egress ACLs.³⁴
- Architecting physical interface assignments, configuring VLAN tagging, and setting up Bridge Virtual Interfaces (BVI) for transparent deployments.²²
- Configuring complex Network Address Translation (NAT) rules to allow internal services to map to public IPs.²⁰

- Tuning internal OS parameters, such as explicitly defining the maximum size of the conntrack state table to prevent state-exhaustion attacks, or modifying TCP retransmission timeout thresholds.⁶

While precise and necessary, manual control is inherently static. If an advanced persistent threat (APT) launches a novel command-and-control campaign from hundreds of rotating IP addresses, manual intervention is far too slow to mitigate the threat effectively. The time required for a human to analyze logs, identify the malicious IPs, and manually type block rules ensures the breach will succeed.

Threat Intelligence Feeds and Automated Ingestion

To bridge the critical gap between static manual rules and highly dynamic threats, modern firewalls must integrate external threat intelligence through automated ingestion. This is achieved via standardized machine-readable protocols, specifically STIX (Structured Threat Information Expression) and TAXII (Trusted Automated eXchange of Indicator Information).⁴⁸ STIX provides a highly structured, standardized syntax to describe cyber threats, mapping the complex relationships between malicious IP addresses, domain names, file hashes, and known threat actor profiles.⁴⁸ TAXII is the transport protocol utilized to securely transmit this STIX-formatted data between global intelligence platforms and the local firewall appliance.⁴⁸ By configuring a firewall to ingest TAXII feeds, the appliance can automatically update its external blocklists in near real-time.⁵⁰ When a global threat research team identifies new adversary infrastructure, the Indicator of Compromise (IoC) is formatted into STIX, distributed via TAXII, and instantaneously enforced by the local firewall engine—completely bypassing the need for a human administrator to manually implement a block rule.⁵⁰

IDS/IPS Integration: Signature and Behavioral Automation

Automated packet evaluation requires the integration of robust Intrusion Detection and Prevention Systems (IDS/IPS). Open-source engines like Snort, Suricata, and Zeek represent the industry standards for deep internal traffic analysis and automated response.⁵²

- **Snort and Suricata:** These platforms operate primarily as signature-based detection engines.⁵² They deeply inspect network payloads against massive, constantly updated databases of known attack patterns (e.g., searching for the specific hexadecimal byte sequences associated with a known remote code execution vulnerability).⁵⁴ Suricata specifically supports multithreading, scaling effectively across modern multi-core processors. When a signature match occurs, these engines can automatically issue an IPS command to the firewall to inject a TCP RST packet to tear down the malicious session, or dynamically drop the source IP temporarily.⁵⁴
- **Zeek (formerly Bro):** Zeek takes a fundamentally different, behavioral-based approach.⁵² Rather than strictly searching for signatures, Zeek acts as a network monitoring framework, parsing all network flows up to Layer 7 and recording highly structured metadata (e.g., DNS queries, HTTP headers, SSL certificate issuer details) into transaction logs.⁵⁴ This anomaly-based detection establishes a baseline of normal network behavior,

allowing automated systems to detect zero-day exploits that lack existing signatures by identifying statistically improbable communication patterns or protocol violations.⁵² In sophisticated Security Operations Centers (SOCs), these engines operate in tandem: Snort or Suricata provides real-time automated blocking for known threats, while Zeek generates the high-fidelity parsed data necessary for advanced threat hunting and behavioral anomaly detection.⁵⁴

Security Orchestration, Automation, and Response (SOAR)

The pinnacle of automated firewall control is the integration of Security Orchestration, Automation, and Response (SOAR) platforms.⁵⁷ SOAR platforms utilize API connectivity to unify disjointed security tools—firewalls, endpoint agents, threat intelligence feeds, and SIEMs (Security Information and Event Management)—into a centralized, programmatic execution environment.⁵⁹

When a firewall or SIEM system detects anomalous activity—such as an internal host attempting unauthorized lateral movement or communicating with an unknown external domain—it generates an alert.⁶⁰ The SOAR platform intercepts this alert and automatically executes a predefined "Playbook".⁵⁹

A SOAR playbook acts as a high-speed, programmatic security analyst. Without requiring human intervention, the playbook can execute a complex workflow:

1. Extract the offending IP address and host details from the firewall alert.⁶⁰
2. Query external threat intelligence APIs (via STIX/TAXII) to check the IP's global reputation and historical context.⁵⁰
3. Instruct an Endpoint Detection and Response (EDR) agent on the infected machine to logically isolate the host from the LAN.⁵⁷
4. Issue an API command directly back to the firewall to dynamically generate and insert a strict containment block rule, cutting off the host's access to the wider internet.⁶⁰

By utilizing SOAR integrations, the Mean Time to Respond (MTTR) to a critical incident is reduced from hours of manual log parsing to mere seconds of automated playbook execution.⁵⁸ Furthermore, machine learning modules within modern SOAR solutions reduce alert fatigue by triaging false positives, presenting human analysts with only the highly contextualized, verified incidents that necessitate qualitative judgment.⁵⁷

The Hybrid Mesh Firewall (HMF) Paradigm

The ultimate evolution of manual configuration and automated synergy is the Hybrid Mesh Firewall (HMF) architecture.⁶² As modern enterprises increasingly deploy workloads across on-premises data centers, multiple public clouds (AWS, Azure), and thousands of remote endpoints, managing isolated, traditional firewalls becomes administratively untenable.¹⁰ The interconnected nature of these environments generates massive volumes of east-west traffic that traditional perimeter firewalls cannot inspect efficiently.⁶³

A Hybrid Mesh Firewall is a unified security platform that coordinates enforcement across hardware appliances, virtual machines, cloud-native firewalls, and Firewall-as-a-Service

(FWaaS) nodes.¹¹ Governed by a centralized management console—often referred to as a "single pane of glass"—an administrator can manually write a global policy, such as restricting access to a specific corporate application based on user identity.⁶²

The underlying HMF engine translates and automatically propagates this manual rule across the diverse environments, ensuring that the technical syntax correctly aligns with the AWS Network Firewall, the on-premise hardware appliance, and the remote user's SASE (Secure Access Service Edge) gateway simultaneously.⁶³ This hybrid orchestration eliminates configuration drift, drastically reduces human configuration errors, and guarantees that automated threat responses—such as IP blocklists generated by SOAR playbooks—are enforced uniformly across the entire distributed organizational perimeter.⁶²

Synthesis and Architectural Conclusion

The architecture of a modern standard firewall is a masterclass in combining highly optimized computational mechanics with intelligent, adaptable operational design. The foundational transition from stateless packet filtering to stateful inspection established the necessary contextual awareness required to manage complex bidirectional communications. The subsequent evolution into Deep Packet Inspection (DPI) granted the firewall application awareness, allowing it to penetrate encrypted TLS 1.3 payloads and combat sophisticated application-layer threats hiding within permitted ports.

To maintain these deep inspections at scale, firewalls rely on complex internal mathematics. Connection tracking engines like Linux conntrack meticulously follow the TCP state machine, generate expiring pseudo-states for connectionless ICMP and UDP traffic, and utilize Application Layer Gateways to dynamically flag negotiated ephemeral ports as related traffic. The logic governing these states is enforced by parsing engines that require administrators to intimately understand whether they are operating within a strict top-down "first match wins" parameter (iptables) or utilizing a declarative "last match wins" configuration optimized by early-exit statements (pf).

As traffic volumes accelerate, traditional kernel-space processing introduces unsustainable context-switching latency. This limitation is actively mitigated by integrating high-performance data planes, utilizing DPDK and eBPF/XDP frameworks to execute filtering logic directly at the network interface hardware. By dropping malicious volumetric traffic before it traverses the operating system stack and avoids `sk_buff` memory allocation, these firewalls maintain line-rate throughput even under duress.

Ultimately, a robust firewall deployment requires an intricate balance of human governance and machine velocity. While the network topology—whether utilizing screened subnets, circuit-level gateways masking internal IPs, or invisible transparent bridged modes—must be deliberately architected by security engineers, the daily operational response cannot rely on manual configuration. Through the seamless ingestion of global threat intelligence via STIX/TAXII protocols, real-time behavioral and signature analysis via Zeek and Suricata, and programmatic API-driven playbook execution via SOAR platforms, firewalls transform into proactive, self-healing enforcement nodes. Coordinated under a unified Hybrid Mesh topology, this comprehensive synthesis of deep technical inspection, manual architectural control, and automated orchestration represents the definitive framework required to secure highly

distributed enterprise environments against an increasingly complex threat landscape.

Works cited

1. Network Firewall stateless and stateful rules engines - AWS Documentation, accessed June 13, 2026, <https://docs.aws.amazon.com/network-firewall/latest/developerguide/firewall-rules-engines.html>
2. What is The Difference Between Stateful & Stateless Firewall? | Fortinet, accessed June 13, 2026, <https://www.fortinet.com/resources/cyberglossary/stateful-vs-stateless-firewall>
3. Understanding Stateful vs Stateless Firewalls for Stateful Protocol ..., accessed June 13, 2026, <https://www.illumio.com/blog/firewall-stateful-inspection>
4. Stateful vs. Stateless Firewall - Check Point Software, accessed June 13, 2026, https://www.checkpoint.com/cyber-hub/network-security/what-is-firewall/what-is-a-stateful-firewall/stateful_vs_stateless_firewall/
5. Stateful vs. Stateless Firewall: Differences Explained | ConnectWise, accessed June 13, 2026, <https://www.connectwise.com/blog/stateful-vs-stateless-firewall>
6. How to Use Connection Tracking (conntrack) with iptables, accessed June 13, 2026, <https://oneuptime.com/blog/post/2026-03-20-iptables-conntrack-connection-tracking/view>
7. How Next-Generation Firewalls Strengthen Enterprise Security Posture - Fortinet, accessed June 13, 2026, <https://www.fortinet.com/resources/cyberglossary/next-generation-firewall>
8. What Is a Next-Generation Firewall (NGFW)? | Zscaler - Zscaler, Inc., accessed June 13, 2026, <https://www.zscaler.com/resources/security-terms-glossary/what-is-next-generation-firewall>
9. Next-Generation Firewall vs. Traditional Firewall - Check Point Software, accessed June 13, 2026, <https://www.checkpoint.com/cyber-hub/network-security/what-is-next-generation-firewall-ngfw/next-generation-firewall-vs-traditional-firewall/>
10. Next-Generation Firewalls (NGFWs): Features, Benefits and Buyer Guide | CDW, accessed June 13, 2026, <https://www.cdw.com/content/cdw/en/articles/security/next-generation-firewalls-complete-ngfw-guide-evaluation.html>
11. Top Enterprise Firewalls Solutions in 2026 - Check Point Software, accessed June 13, 2026, <https://www.checkpoint.com/de/cyber-hub/tools-vendors/top-enterprise-firewalls-solutions-in-2026/>
12. Chapter 7. The state machine, accessed June 13, 2026, <https://lewestech.com/mirrors/www.iptables.info/en/connection-state.html>
13. Connection Tracking (conntrack): Design and Implementation Inside Linux Kernel, accessed June 13, 2026,

- <https://arthurchiao.art/blog/conntrack-design-and-implementation/>
14. What is ICMP? ICMP Protocol Explained - AWS, accessed June 13, 2026,
<https://aws.amazon.com/what-is/icmp/>
 15. 5.11. Managing ICMP Requests | Security Guide | Red Hat Enterprise Linux | 7,
accessed June 13, 2026,
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-managing_icmp_requests
 16. What is ICMP (Internet Control Message Protocol)? - Fortinet, accessed June 13, 2026,
<https://www.fortinet.com/resources/cyberglossary/internet-control-message-protocol-icmp>
 17. Does the connection tracking really necessary for receiving ICMP echo request packets?, accessed June 13, 2026,
https://www.reddit.com/r/linuxadmin/comments/1bg0jvj/does_the_connection_tracking_really_necessary_for/
 18. A Novice's Guide to Crafting a Linux Firewall with XDP and a ..., accessed June 13, 2026, <https://blogs.oracle.com/linux/ebpf-firewall-xdp>
 19. Your Complete CISSP Study Guide to Firewall Types, Architectures ..., accessed June 13, 2026, <https://destcert.com/resources/firewall-types-architectures/>
 20. Firewall types and architecture - Infosec Institute, accessed June 13, 2026,
<https://www.infosecinstitute.com/resources/network-security-101/firewall-types-and-architecture/>
 21. Understanding the main firewall topologies - OSTEC Blog, accessed June 13, 2026,
<https://ostec.blog/en/perimeter/understanding-the-main-firewall-topologies/>
 22. What Is a Transparent Firewall? - Fortinet, accessed June 13, 2026,
<https://www.fortinet.com/resources/cyberglossary/transparent-firewall>
 23. Cisco Secure Firewall Modes - The Network DNA, accessed June 13, 2026,
<https://www.thenetworkdna.com/2024/03/cisco-secure-firewall-modes.html>
 24. CLI Book 1: Cisco Secure Firewall ASA Series General Operations CLI Configuration Guide, 9.19 - Transparent or Routed Firewall Mode [Cisco Secure Firewall ASA], accessed June 13, 2026,
<https://www.cisco.com/c/en/us/td/docs/security/asa/asa919/configuration/general/asa-919-general-config/intro-fw.html>
 25. Transparent firewalls : r/networking - Reddit, accessed June 13, 2026,
https://www.reddit.com/r/networking/comments/cr0z7x/transparent_firewalls/
 26. Firepower Management Center Configuration Guide, Version 6.6 - Transparent or Routed Firewall Mode for Firepower Threat Defense [Cisco Secure Firewall Management Center], accessed June 13, 2026,
https://www.cisco.com/c/en/us/td/docs/security/firepower/660/configuration/guide/fpmc-config-guide-v66/transparent_or_routed_firewall_mode_for_firepower_threat_defense.html
 27. What Is a Circuit Level Gateway? [+ How It Works & Examples] - Palo Alto Networks, accessed June 13, 2026,
<https://www.paloaltonetworks.com/cyberpedia/what-is-a-circuit-level-gateway>

28. Circuit-Level Gateways: Definition, Features & Examples - Datamation, accessed June 13, 2026,
<https://www.datamation.com/networks/circuit-level-gateways-definition-features-examples/>
29. What Is a Circuit-Level Gateway? How They Work and When to Use One - Enterprise Networking Planet, accessed June 13, 2026,
<https://www.enterprisenetworkingplanet.com/security/circuit-level-gateway/>
30. SOCKS - Wikipedia, accessed June 13, 2026, <https://en.wikipedia.org/wiki/SOCKS>
31. RFC 1928 - SOCKS Protocol Version 5 - Datatracker - IETF, accessed June 13, 2026, <https://datatracker.ietf.org/doc/html/rfc1928>
32. What Is a Proxy Firewall and How Does It Work? - Fortinet, accessed June 13, 2026, <https://www.fortinet.com/resources/cyberglossary/proxy-firewall>
33. Is there any difference between circuit vs application level gateway except that one operates in layer 3 and another at layer 7 of OSI model? - Information Security Stack Exchange, accessed June 13, 2026,
<https://security.stackexchange.com/questions/262900/is-there-any-difference-between-circuit-vs-application-level-gateway-except-that>
34. The Intricacies of Firewall Rules Order: Navigating Best Practices - Tufin, accessed June 13, 2026,
<https://www.tufin.com/blog/firewall-rules-order-intricacies-navigating-best-practices>
35. What is the order that the firewall processes the firewall rules and settings - Broadcom TechDocs, accessed June 13, 2026,
<https://techdocs.broadcom.com/us/en/symantec-security-software/endpoint-security-and-management/endpoint-protection/all/Using-policies-to-manage-security/managing-firewall-protection-v15703943-d53e474/managing-firewall-rules-v44982524-d53e169/about-the-firewall-rule-firewall-setting-and-intru-v8642294-d53e338.html>
36. Best Firewall Solutions for FreeBSD in 2026 | FreeBSD Software, accessed June 13, 2026, <https://freebsdsoftware.org/blog/best-firewall-freebsd/>
37. Azure Firewall rule processing logic | Microsoft Learn, accessed June 13, 2026, <https://learn.microsoft.com/en-us/azure/firewall/rule-processing>
38. ipf use dhcp/pppoe interface ip in ruleset - The FreeBSD Forums, accessed June 13, 2026,
<https://forums.freebsd.org/threads/ipf-use-dhcp-pppoe-interface-ip-in-ruleset.93004/>
39. The pfSense Book - DOKUMEN.PUB, accessed June 13, 2026,
<https://dokumen.pub/the-pfsense-book.html>
40. Managing evaluation order for Suricata compatible rules in AWS Network Firewall, accessed June 13, 2026,
<https://docs.aws.amazon.com/network-firewall/latest/developerguide/suricata-rule-evaluation-order.html>
41. Userspace vs Kernelspace: Understanding the Divide | linux - Oracle Blogs, accessed June 13, 2026,
<https://blogs.oracle.com/linux/userspace-vs-kernelspace-understanding-the-divi>

[de](#)

42. Performance of Kernel space vs User space written program? : r/linux - Reddit, accessed June 13, 2026, https://www.reddit.com/r/linux/comments/sjnjsy/performance_of_kernel_space_vs_user_space_writen/
43. Fundamentals of P4 and DPDK - University of South Carolina, accessed June 13, 2026, <https://research.cec.sc.edu/files/cyberinfra/files/1--fundamentals-of-p4-and-dpdk-1.pdf>
44. Man page of NFT - Netfilter.org, accessed June 13, 2026, <https://www.netfilter.org/projects/nftables/manpage.html>
45. Chapter 3. Getting started with XDP and eBPF | Configuring firewalls and packet filters | Red Hat Enterprise Linux, accessed June 13, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_firewalls_and_packet_filters/getting-started-with-xdp-and-ebpf
46. eBPF XDP: The Basics and a Quick Tutorial | Tigera – Creator of Calico, accessed June 13, 2026, <https://www.tigera.io/learn/guides/ebpf/ebpf-xdp/>
47. Understanding Delays in AF_XDP-based Applications - arXiv, accessed June 13, 2026, <https://arxiv.org/html/2402.10513v1>
48. Complete Guide to Threat Intelligence Feeds in 2026 | CyCognito, accessed June 13, 2026, <https://www.cycognito.com/learn/threat-intelligence/threat-intelligence-feeds/>
49. What are STIX/TAXII (Structured Threat Information eXpression / Trusted Automated eXchange of Intelligence Information) Standards? | Anomali, accessed June 13, 2026, <https://www.anomali.com/glossary/stix-taxii>
50. Improve Global Threat Detection Using STIX and TAXII within Silent Push, accessed June 13, 2026, <https://www.silentpush.com/blog/stix-and-taxii/>
51. Implementing Threat Intelligence Integration for Enterprise Security - Deepwatch, accessed June 13, 2026, <https://www.deepwatch.com/glossary/threat-intelligence-integration/>
52. What is the Difference Between Snort and Zeek? - Stamus Networks, accessed June 13, 2026, <https://www.stamus-networks.com/blog/what-is-the-difference-between-snort-and-zeek>
53. Evaluating the Efficacy of Network Forensic Tools: A Comparative Analysis of Snort, Suricata, and Zeek in Addressing Cyber Vulnerabilities - SANS Institute, accessed June 13, 2026, <https://www.sans.org/white-papers/evaluating-efficacy-of-network-forensic-tools-comparative-analysis-snort-suricata-zeek-addressing-cyber-vulnerabilities>
54. Snort vs Zeek. In network security, monitoring what... | by Huseyn Eyvazov | Medium, accessed June 13, 2026, <https://medium.com/@heyvazov15/snort-vs-zeek-1d0b548f11ae>
55. Which open source IDS is easiest to observe logs and create rules? : r/cybersecurity - Reddit, accessed June 13, 2026, https://www.reddit.com/r/cybersecurity/comments/18dl4iz/which_open_source_id

[s_is_easiest_to_observe_logs/](#)

56. A novel hybrid-based approach of snort automatic rule generator and security event correlation (SARG-SEC) - PMC, accessed June 13, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9044335/>
57. What is SOAR (security orchestration, automation and response)? - IBM, accessed June 13, 2026,
<https://www.ibm.com/think/topics/security-orchestration-automation-response>
58. What Is SOAR? Security Orchestration, Automation, and Response - Fortinet, accessed June 13, 2026,
<https://www.fortinet.com/resources/cyberglossary/what-is-soar>
59. What Is SOAR? - Palo Alto Networks, accessed June 13, 2026,
<https://www.paloaltonetworks.com/cyberpedia/what-is-soar>
60. Integrating Firewalls with SIEM and SOAR Platforms for Automated Threat Response, accessed June 13, 2026,
<http://eprints.umsida.ac.id/16143/1/363%20Integrating%20Firewalls%20with%20SIEM%20and%20SOAR%20Platforms%20for%20Automated%20Threat%20Response.pdf>
61. Embracing SOAR for Effective Threat Hunting | Tufin, accessed June 13, 2026,
<https://www.tufin.com/blog/embracing-soar-effective-threat-hunting>
62. Hybrid Mesh Firewall - Fortinet, accessed June 13, 2026,
<https://www.fortinet.com/solutions/hybrid-mesh-firewall>
63. What Is A Hybrid Mesh Firewall? - Fortinet, accessed June 13, 2026,
<https://www.fortinet.com/resources/cyberglossary/hybrid-mesh-firewall>
64. Fortinet Next-Generation Firewall versus Palo Alto Networks NGFW, accessed June 13, 2026,
<https://www.fortinet.com/products/next-generation-firewall/fortigate-vs-pan>